

Rețele Petri Stohastice - Model de simulare pentru evaluarea performanțelor în sisteme de servire cu topologie complexă

Emanuela Cucinschi

Cuprins

1	Introducere	3
1.1	Contextul temei și motivația	3
1.2	Obiectivele lucrării	3
1.3	Structura lucrării	4
1.4	Metodologia utilizată	4
2	Fundamentarea teoretică și soluții similare	5
2.1	Sisteme cu evenimente discrete	5
2.2	Rețele Petri	5
2.3	Rețele Petri stohastice	6
2.4	Sisteme de servire și indicatori de performanță	7
2.5	Modele și soluții existente	7
2.6	Limitări ale soluțiilor existente și cerințele modelului propus	7
3	Soluția propusă	8
3.1	Formularea problemei și strategia de abordare	8
3.2	Arhitectura sistemului	8
3.3	Algoritmi și Pseudocod	11
3.4	Alegerea tehnologiei	14
3.5	Descrierea implementării	14
4	Testarea soluției și rezultate experimentale	16
4.1	Metodologia de testare	16
4.2	Modelul în serie	16
4.3	Modelul paralel	18
4.4	Modelul sincron	19
5	Concluzii și dezvoltări ulterioare	23
5.1	Concluzii generale	23
5.2	Sinteza rezultatelor și contribuții personale	23

Rezumat

Lucrarea tratează evaluarea performanțelor în sistemele de servire cu topologie complexă, folosind formalismul Rețelelor Petri Stohastice.

Această temă a fost aleasă deoarece rețelele Petri stohastice oferă un cadru eficient pentru modelarea și simularea sistemelor de servire complexe, în care evenimentele se desfășoară concurent, iar timpii de execuție sunt variabili. Cu ajutorul rețelelor Petri stohastice poți observa: cât de aglomerat devine un sistem, unde apar blocajele, cât timp așteaptă un client, câte resurse sunt necesare, ce se întâmplă dacă adaugi un server nou, cum se schimbă performanța sistemului când crește numărul de cereri, cât de eficient este sistemul în diferite scenarii. Prin utilizarea acestui model se pot evalua performanțe precum timpul mediu de așteptare, gradul de utilizare a resurselor și apariția blocajelor.

Analiza acestor sisteme este importantă deoarece permite testarea mai multor scenarii înainte de implementarea lor în realitate, contribuind la optimizarea proceselor și la creșterea eficienței în domenii precum transportul, medicina, producția, comunicațiile și serviciile publice.

Metodologia utilizată în cadrul lucrării constă în parcurgerea mai multor etape logice, pornind de la studiul teoretic al rețelelor Petri și al rețelelor Petri stohastice, continuând cu alegerea unui sistem de servire care poate fi modelat și analizat. Pe baza acestui sistem au fost identificate componentele principale, precum clienții, cozile de așteptare, serverele și etapele de procesare.

Ulterior, sistemul a fost reprezentat sub forma unei rețele Petri stohastice, folosind locuri, tranziții, arce și jetoane. Componenta stohastică a fost introdusă prin asocierea unor timpi variabili tranzițiilor, astfel încât modelul să fie cât mai apropiat de comportamentul unui sistem real.

După construirea modelului, acesta a fost simulat pentru a observa evoluția sistemului în timp. Rezultatele obținute au fost analizate din punctul de vedere al performanței, urmărind indicatori precum timpul de așteptare, gradul de utilizare a resurselor și apariția eventualelor blocaje. Pe baza acestor rezultate au fost formulate concluzii privind eficiența sistemului și posibile direcții de îmbunătățire.

Pentru evaluare sunt urmăriți indicatori precum timpul mediu de răspuns și lungimea maximă a cozilor de așteptare. Metodologia utilizată are la bază simularea cu evenimente discrete, iar implementarea simulatorului este realizată în limbajul C++. Soluția propusă permite modelarea timpilor de sosire și a timpilor de servire cu ajutorul unor distribuții probabilistice, în special distribuția exponențială și distribuția normală. Rezultatele obținute evidențiază faptul că performanțele sistemului depind atât de topologia utilizată, cât și de variația timpilor de prelucrare. Reducerea dispersiei timpilor de servire conduce, în general, la scăderea timpului mediu de răspuns și la o funcționare mai stabilă a sistemului analizat.

Capitolul 1. Introducere

Sistemele de servire cu topologie complexă apar frecvent în domenii precum sistemele de calcul paralele, rețelele de comunicații, sistemele distribuite și aplicațiile în care mai multe cereri folosesc resurse comune. În aceste sisteme, evaluarea performanțelor este importantă încă din faza de proiectare, deoarece oferă posibilitatea de a analiza comportamentul și evoluția sistemului în diferite condiții, înainte de implementare sau utilizare practică. Indicatorii utilizați pentru evaluarea performanței, precum timpul mediu de răspuns, lungimea cozilor de așteptare și gradul de utilizare a resurselor, oferă informații esențiale despre eficiența și stabilitatea sistemului analizat.

Tema lucrării propune dezvoltarea unui simulator software bazat pe Rețele Petri Stohastice, utilizat pentru evaluarea performanțelor în sisteme de servire cu topologii diferite. Sunt analizate modele de organizare în serie, în paralel și modele cu sincronizare, urmărindu-se modul în care structura sistemului și distribuția timpilor de servire influențează timpul mediu de răspuns și lungimea cozilor de așteptare.

1.1. Contextul temei și motivația

În prezent, multe sisteme reale sunt formate din procese care se desfășoară în paralel și care implică numeroase elemente ce interacționează între ele, precum: resurse, date, procese și reguli de funcționare. Astfel de sisteme pot fi întâlnite în domenii precum transportul, comunicațiile, producția, serviciile publice sau sistemele informatice.

Deoarece un sistem real este permanent dependent de elementele din care este format și de factorii externi care influențează funcționarea acestuia, devine esențială identificarea și evaluarea unor metode capabile să îmbunătățească performanța acestor sisteme, vizând direct minimizarea timpilor de așteptare, gestionarea optimă a resurselor și sporirea vitezei de execuție.

Pentru studiul unor astfel de arhitecturi complexe, metodele analitice clasice devin limitate, în special atunci când intervin mecanisme de concurență sau partajare a resurselor. Din acest motiv, este necesară o abordare de modelare capabilă să descrie clar dinamica proceselor. În acest sens, formalismul Rețelelor Petri reprezintă un instrument optim, deoarece nu surprinde doar stările finale, ci evidențiază cu exactitate regulile și condițiile care determină evoluția întregului sistem.

Mai mult, pentru a reflecta fidel realitatea — în care apar frecvent cozi de așteptare, utilizare neuniformă a resurselor și timpi variabili de execuție —, se recurge la Rețelele Petri Stohastice. Acestea extind modelul de bază prin introducerea factorului probabilistic, reușind astfel să surprindă mult mai fidel funcționarea sistemelor reale și să valideze corect soluțiile de optimizare.

1.2. Obiectivele lucrării

Această lucrare urmărește utilizarea Rețelelor Petri Stohastice pentru simularea și evaluarea performanțelor unor sisteme de servire cu topologie complexă.

Obiectivul general al cercetării este dezvoltarea unui mediu de simulare care să permită modelarea, testarea și compararea mai multor tipuri de sisteme de servire. Pentru atingerea acestui scop, obiectivele specifice urmărite sunt:

- construirea unor modele care descriu cum sunt procesate cererile/ clienții într-un sistem;
- implementarea unui simulator cu evenimente discrete în limbajul C++;
- modelarea timpilor de sosire și de servire prin variații probabilistice;

- analizarea comportamentului sistemelor organizate în serie, paralel și sincron;
- compararea rezultatelor obținute pentru distribuția exponențială și distribuția normală a fiecărui tip de sistem analizat;
- identificarea influenței dispersiei timpilor de prelucrare asupra performanțelor sistemului;
- analizarea performanței sistemului prin urmărirea timpilor de așteptare, a utilizării resurselor și a posibilelor blocaje apărute în timpul simulării.

1.3. Structura lucrării

Prezenta lucrare este organizată în cinci capitole, structurate astfel încât să asigure o tranziție logică de la conceptele teoretice la validarea experimentală a soluției propuse:

Capitolul 1 prezintă introducerea, contextul temei, motivația alegerii acesteia, obiectivele urmărite prin prezenta cercetare și structura generală a lucrării.

Capitolul 2 conține fundamentarea teoretică necesară înțelegerii modelelor dezvoltate, abordând concepte de bază privind sistemele de servire, precum și teoria Rețelelor Petri și a Rețelelor Petri Stohastice.

Capitolul 3 detaliază soluția propusă, descriind componentele sistemului, arhitectura modelelor (serie, paralel, sincron) și modul în care acestea sunt reprezentate în cadrul rețelei Petri stohastice.

Capitolul 4 este dedicat simulării modelului și prezentării rezultatelor experimentale, incluzând metodologia de testare și analiza indicatorilor de performanță obținuți.

Capitolul 5 sintetizează concluziile finale desprinse în urma analizei rezultatelor și propune posibile direcții de dezvoltare viitoare ale aplicației și modelelor studiate.

1.4. Metodologia utilizată

Metodologia utilizată în această lucrare se bazează pe simularea cu evenimente discrete, o tehnică adecvată pentru modelarea sistemelor în care schimbările de stare se produc instantaneu, la momente specifice de timp (precum sosirea unei cereri sau finalizarea unei prelucrări).

Pentru a evalua performanța topologiilor propuse, metodologia de lucru a urmat un proces structurat în mai multe etape:

1. **Definirea sistemelor de servire:** S-au identificat și selectat arhitecturile de interes (modelele în serie, paralel și sincron), stabilindu-se componentele esențiale ale acestora și modul de interacțiune.
2. **Modelarea formală:** Sistemele identificate au fost transpuse în modele matematice riguroase utilizând formalismul rețelelor Petri, prin definirea precisă a locurilor, tranzițiilor, arcelor de conectare și a distribuției inițiale a jetoanelor.
3. **Integrarea factorului temporal:** Pentru a reflecta comportamentul real, modelelor le-au fost atașați timpi stohastici de execuție asociați tranzițiilor, transformându-le astfel în rețele Petri temporizate.
4. **Implementarea și execuția simulărilor:** Modelele au fost rulate în cadrul simulatorului software dezvoltat, parametrii de intrare (precum ratele de sosire și de servire) fiind variați controlat pentru a forța sistemele să tranziteze de la regimul stabil către pragul de saturație.
5. **Analiza și interpretarea datelor:** Ultima etapă a constat în colectarea indicatorilor (timpul mediu de răspuns, dimensiunea maximă a cozilor) și analiza lor comparativă, pentru a formula concluzii fundamentate privind eficiența fiecărei topologii.

Capitolul 2. Fundamentarea teoretică și soluții similare

Rețelele Petri reprezintă un model matematic puternic pentru descrierea și simularea sistemelor cu evenimente discrete. Ele nu exclud metodele clasice de analiză, ci le completează, facilitând înțelegerea comportamentului dinamic al sistemelor complexe. Acest instrument devine esențial atunci când arhitectura analizată implică procese concurente, sincronizare între resurse și partajarea acestora, contexte în care declanșarea evenimentelor prezintă un caracter nedeterminist.

2.1. Sisteme cu evenimente discrete

Un sistem cu evenimente discrete se caracterizează printr-o stare care nu evoluează continuu în timp, ci se modifică punctual, prin salturi instantanee, exclusiv la apariția unor evenimente bine definite. Spre deosebire de sistemele continue, modelate prin ecuații diferențiale, în cazul sistemelor cu evenimente discrete starea rămâne constantă între două tranziții succesive (de exemplu, nu există stări intermediare de „procesare parțială”). Astfel, evoluția sistemului este dictată strict de natura evenimentelor declanșatoare și de momentele producerii acestora.

Într-o arhitectură de servire, evenimentele discrete tipice includ sosirea unei cereri, inițierea procesării, finalizarea servirii sau eliberarea unei resurse partajate. Această structură conceptuală se regăsește într-o gamă diversificată de aplicații din lumea reală, precum rețelele de calculatoare, sistemele de procesare paralelă sau liniile de producție. Din acest motiv, studiul sistematic al acestor arhitecturi prezintă o relevanță practică majoră pentru minimizarea timpilor de așteptare și optimizarea utilizării resurselor.

Pentru a înțelege complet comportamentul acestor arhitecturi, este esențială distincția dintre modul în care se schimbă starea sistemului și momentul declanșării acestor schimbări. Conceptul de „eveniment discret” definește exclusiv *cum* se modifică starea. În schimb, natura deterministă sau stohastică a proceselor definește *când* au loc aceste evenimente. Dacă momentele de apariție ale evenimentelor ar fi cunoscute cu certitudine absolută (de exemplu, sosiri la intervale fixe de timp), sistemul ar fi unul determinist.

În practică, însă, o asemenea certitudine este aproape imposibil de atins. Timpii de sosire și duratele de execuție fluctuează, fiind dependenți de o multitudine de factori, ceea ce transformă ansamblul într-un sistem *stochastic*. Prin urmare, arhitecturile studiate în această lucrare se încadrează simultan în ambele categorii: sunt cu evenimente discrete (deoarece stările se modifică prin tranziții punctuale) și stohastice (deoarece momentele declanșării evenimentelor sunt generate pe baza unor variabile aleatoare).

Evaluarea riguroasă a performanțelor unor astfel de sisteme complexe impune utilizarea simulatoarelor cu evenimente discrete și a unor metode probabilistice de analiză matematică, instrumente ce vor fi detaliate în secțiunile următoare ale acestui capitol.

2.2. Rețele Petri

O rețea Petri este, în forma sa cea mai simplă, un graf orientat cu două categorii de noduri — locațiile (*places*) și tranzițiile (*transitions*) — între care sunt permise arce doar de la un tip de nod la celălalt, niciodată între două noduri de același tip [4].

Fiecărui astfel de graf i se asociază un vector de stare, denumit vector marcaj. Conform definiției, „la modelarea unui proces cu evenimente discrete, o tranziție reprezintă un anumit tip de eveniment, în timp ce locațiile și vectorul marcaj descriu condițiile în care evenimentele considerate se pot produce” [6].

În ceea ce privește comportamentul rețelei, declanșarea unei tranziții validate presupune eliminarea

jetoanelor (*tokens*) din locațiile de intrare și adăugarea lor în locațiile de ieșire, reproducând astfel trecerea sistemului dintr-o stare în alta. Pentru a ilustra acest concept, într-o arhitectură de servire cu o singură stație, o locație poate reprezenta coada de așteptare, în care fiecare jeton corespunde unei cereri. Tranziția asociată începerii servirii devine executabilă exclusiv în momentul în care există cel puțin un jeton disponibil în acea locație.

Din punct de vedere matematic, o rețea Petri netemporizată reprezintă un cvintuplu $PN = (P, T, I, O, M_0)$, având următoarele componente:

- P este mulțimea finită a locațiilor;
- T este mulțimea finită a tranzițiilor, cu condiția ca $P \cap T = \emptyset$ (mulțimi disjuncte);
- $I : P \times T \rightarrow \{0, 1, 2, \dots\}$ reprezintă funcția de incidență înainte, care definește ponderea arcelor orientate de la locații către tranziții;
- $O : T \times P \rightarrow \{0, 1, 2, \dots\}$ reprezintă funcția de incidență înapoi, care definește ponderea arcelor orientate de la tranziții către locații;
- $M_0 : P \rightarrow \{0, 1, 2, \dots\}$ este marcajul inițial al rețelei.

Dinamica rețelei este determinată de regula tranziției. O tranziție $t \in T$ este considerată validată într-un anumit marcaj M dacă și numai dacă $M(p) \geq I(p, t)$ pentru orice locație de intrare $p \in P$. Executarea unei tranziții validate generează un nou marcaj M' , calculat conform ecuației de stare:

$$M'(p) = M(p) - I(p, t) + O(t, p), \quad \forall p \in P$$

Această regulă fundamentală a validării și executării tranzițiilor stă la baza mecanismului de antrenare a rețelei, constituind nucleul algoritmilor de simulare detaliați în Capitolul 3.

2.3. Rețele Petri stohastice

Modelul de bază al rețelelor Petri surprinde doar ordinea posibilă de producere a evenimentelor, fără nicio informație despre durata lor. Pentru a putea evalua cantitativ un sistem — nu doar dacă o anumită secvență de evenimente este posibilă, ci și cât timp durează — este necesară o extensie temporizată a modelului, în care tranzițiilor li se asociază durate.

Când aceste durate nu sunt constante, ci variabile aleatoare — de exemplu, pentru că timpul de servire diferă de la o cerere la alta — rețeaua este numită stohastică [2]. Această variabilitate este, de altfel, o proprietate caracteristică a sistemelor reale de servire: doi clienți succesivi rareori necesită exact același timp de procesare, iar un model care ar ignora această variație ar simplifica excesiv comportamentul real al sistemului.

Alegerea distribuției exponențiale pentru modelarea duratelor aleatoare nu este întâmplătoare: pe lângă faptul că reflectă rezonabil variabilitatea proceselor de sosire și de servire din multe sisteme reale, proprietatea de lipsă de memorie a acestei distribuții face ca graful stărilor accesibile al rețelei, obținut prin antrenarea ei, să coincidă cu graful unui lanț Markov echivalent — rețeaua Petri devenind astfel și un instrument de construcție asistată a modelului Markov corespunzător [7]. Această corespondență este exploatată explicit în capitolele următoare, unde ratele de sosire și de servire ale sistemului analizat sunt eșantionate din distribuții exponențiale.

Pe lângă distribuția exponențială — utilă datorită proprietății de lipsă de memorie și compatibilității cu lanțurile Markov —, în modelarea stohastică se utilizează frecvent și **distribuția normală (gaussiană)**. Ea permite reprezentarea proceselor ale căror durate sunt simetrice față de o valoare medie, fiind utilă în analiza impactului pe care dispersia și fluctuațiile controlate ale timpilor de servire le au asupra performanței generale. În cadrul simulatorului dezvoltat pentru această lucrare, ambele modele de distribuție au fost implementate și evaluate comparativ, urmărindu-se modul în care comportamentul stohastic influențează indicatorii de performanță ai sistemului.

2.4. Sisteme de servire și indicatori de performanță

Un sistem de servire este caracterizat, în esență, de un flux de cereri care sosesc conform unui anumit tipar, de una sau mai multe stații care le prelucreează, de o disciplină de servire (de exemplu, principiul primul sosit, primul servit — FIFO) și de regulile după care cererile părăsesc sistemul odată procesate.

Pentru a evalua performanța unui asemenea sistem se urmăresc, de regulă, câțiva indicatori standard: timpul mediu de așteptare, timpul mediu total petrecut de o cerere în sistem, lungimea medie și maximă a cozilor de așteptare, gradul de utilizare a fiecărei stații de servire și numărul de cereri procesate într-un interval dat. Aceștia constituie setul principal de parametri analizați în capitolul de rezultate experimentale al prezentei lucrări, pentru fiecare dintre topologiile evaluate.

O notație consacrată pentru descrierea succintă a unui sistem de servire este cea introdusă de Kendall [8], de forma $A/B/c$, unde A descrie procesul de sosire, B procesul de servire, iar c numărul de servere.

2.5. Modele și soluții existente

Pentru analiza sistemelor cu evenimente discrete există, în literatura de specialitate, mai multe abordări consacrate.

Modelele Markov oferă un cadru analitic riguros, bazat pe graful stărilor sistemului și pe ratele de tranziție dintre acestea; problema principală apare atunci când sistemul modelat are mai multe componente care pot evolua independent, caz în care numărul stărilor posibile crește exponențial — fenomen cunoscut ca *explozia spațiului de stări*. Teoria clasică a cozilor de așteptare (notația Kendall, de exemplu $M/M/1$ sau $M/M/c$) oferă formule analitice pentru sisteme cu o structură simplă, dar devine greu de aplicat pentru topologii cu mai multe etape, resurse partajate sau reguli de prioritate.

Rețelele Petri stohastice oferă un compromis util între cele două abordări: spre deosebire de un lanț Markov construit direct, o rețea Petri păstrează o reprezentare grafică explicită a structurii sistemului, care facilitează documentarea și identificarea vizuală a resurselor critice, permițând, în același timp, o evaluare cantitativă riguroasă bazată pe lanțuri Markov.

2.6. Limitări ale soluțiilor existente și cerințele modelului propus

Deși riguroase din punct de vedere matematic, abordările analitice standard prezintă limitări de scalabilitate și flexibilitate, motivând astfel opțiunea propusă în cadrul acestei lucrări. Modelele Markov devin rapid impracticabile pentru sisteme cu mai multe componente concurente, din cauza exploziei spațiului de stări menționate mai sus. Teoria cozilor de așteptare oferă rezultate analitice exacte doar pentru topologii relativ simple, majoritatea rezultatelor clasice presupunând o singură stație sau structuri foarte regulate, ceea ce le face greu de extins la sisteme cu prioritate între clase de cereri sau cu sincronizare între mai multe resurse. În plus, niciuna dintre cele două abordări nu oferă, prin ea însăși, o reprezentare grafică a structurii sistemului, utilă atât pentru comunicarea modelului, cât și pentru identificarea intuitivă a punctelor critice.

Aceste limitări motivează alegerea rețelelor Petri stohastice ca instrument de modelare și simulare pentru sistemul cu topologie complexă analizat în capitolele următoare. Pornind de la ele, modelul propus trebuie să îndeplinească următoarele cerințe: să reprezinte un sistem de servire cu mai multe etape de prelucrare și mai multe clase de cereri; să modeleze explicit sosirea cererilor, formarea cozilor de așteptare și disponibilitatea resurselor de servire; să permită introducerea unor timpi de sosire și de servire variabili, modelați ca variabile aleatoare; să susțină o regulă de prioritate între clasele de cereri care concurează pentru aceeași resursă; și să permită modelarea unei etape de sincronizare, în care finalizarea unei cereri depinde de finalizarea simultană a mai multor ramuri de procesare.

Capitolul 3 detaliază modul concret în care aceste cerințe au fost transpuse într-o arhitectură de rețea Petri stohastică, precum și algoritmul de simulare folosit pentru evaluarea ei.

Capitolul 3. Soluția propusă

În acest capitol este prezentată arhitectura modelului propus, detaliindu-se structura fluxului de date, mecanismele de sincronizare și interacțiunea dintre componentele stohastice ale sistemului simulat.

3.1. Formularea problemei și strategia de abordare

Sistemele actuale cu evenimente discrete presupun existența unor procese concurente complexe, care împart resurse și necesită mecanisme riguroase de sincronizare. Dificultatea majoră în optimizarea acestora provine din natura aleatoare a parametrilor temporali: variația timpilor de sosire a cererilor și a duratelor de servire generează frecvent blocaje, o utilizare inefficientă a resurselor sau acumularea unor cozi de așteptare supradimensionate.

Deoarece testarea directă pe un sistem fizic implică riscuri și costuri ridicate, se impune utilizarea modelării matematice și a simulării. Pentru acest tip de analiză, rețelele Petri oferă un cadru matematic consacrat, capabil să surprindă atât comportamentul calitativ, cât și pe cel cantitativ al sistemelor cu evenimente discrete [5]. Prin extinderea modelului clasic cu timpi aleatori de execuție, varianta stohastică permite o evaluare cantitativă detaliată a performanțelor sistemului, dincolo de simpla validare structurală [2].

Strategia de cercetare și implementare urmărește trei etape logice:

1. **Modelarea arhitecturală:** Proiectarea a trei topologii fundamentale, des întâlnite în arhitecturile sistemelor reale de servire: modelul cu organizare în serie, modelul paralel și modelul sincron.
2. **Implementarea software:** Transpunerea modelelor teoretice într-un mediu de programare adecvat simulării sistemelor cu evenimente discrete. Modelul de simulare va utiliza generatoare de numere pseudo-aleatoare pentru a asocia tranzițiilor timpi de întârziere bazați pe distribuții probabilistice exponențiale (caracterizate de ratele de sosire λ și de servire μ).
3. **Analiza performanțelor:** Executarea simulărilor pe durate extinse pentru a colecta și interpreta metrici critice, cum ar fi timpul mediu de răspuns și dimensiunea maximă a cozilor de așteptare, determinând astfel condițiile de stabilitate pentru fiecare arhitectură în parte.

3.2. Arhitectura sistemului

Pentru a putea simula și analiza performanțele, procesele reale sunt transpuse în arhitectura specifică rețelelor Petri stohastice. Elementele fizice și logice ale sistemelor de servire au fost mapate pe componentele fundamentale ale grafurilor, astfel:

- **Locațiile (Places) și Jetoanele (Tokens):** Locațiile modelează stările posibile ale sistemului (ex. cozi de așteptare, buffere, starea de disponibilitate a unui server), iar marcajul curent — distribuția jetoanelor pe locații — indică în orice moment care dintre aceste stări este activă. Jetoanele reprezintă entitățile dinamice (sarcini, cereri) care traversează sistemul [5, 4].
- **Tranzițiile (Transitions):** Modelează evenimentele discrete propriu-zise; executarea unei tranziții validate produce o nouă distribuție a jetoanelor, echivalentă cu o schimbare de stare a sistemului real [5, 4]. Pentru a putea cuantifica performanțele, s-a folosit extinderea temporizată a formalismului, în care fiecărei tranziții relevante i se atașează o durată — timpul care trebuie să treacă din momentul validării până la executarea ei efectivă [2]. Întrucât aceste durate sunt

eșantionate din distribuții probabilistice (parametrizate prin λ pentru sosiri și μ pentru servire), rezultă o rețea Petri stohastică [2].

Pentru a acoperi principalele moduri de interacțiune din sistemele concurente, arhitectura propusă analizează trei modele topologice distincte:

1. Modelul cu organizare în serie

Din punct de vedere structural, modelul poate fi descris ca o înlănțuire a două subsisteme de servire cu o singură stație fiecare, fără nicio restricție asupra numărului de cereri aflate simultan în sistem [6]. Fluxul de procesare este strict secvențial: o cerere nu este considerată finalizată la primul nod decât în momentul în care devine disponibilă pentru al doilea, cele două evenimente fiind, de fapt, unul singur în model [6]. La nivel de rețea, un jeton parcurge succesiv locațiile și tranzițiile; intervalele dintre sosiri și duratele celor două etape de procesare sunt generate ca variabile aleatoare exponențiale, de parametri λ , respectiv μ_1 și μ_2 [7, 6].

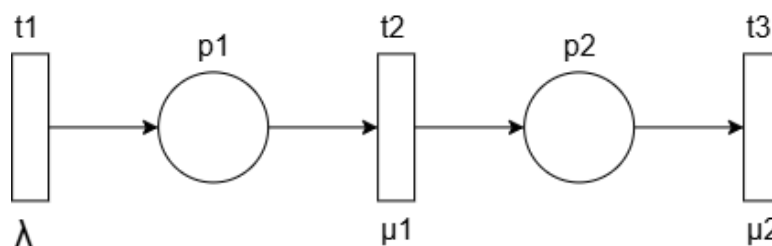


Figura 3.1: Modelul serie al rețelei Petri

2. Modelul cu organizare paralelă

Spre deosebire de organizarea în serie, aici cele două procesoare nu mai depind unul de celălalt în timpul prelucrării: fiecare primește, în același moment, câte o componentă a aceleiași cereri pereche și o tratează independent, în ritm propriu [6]. Această independență introduce însă o problemă de sincronizare la final — cererea nu poate fi considerată încheiată decât după ce ambele componente au fost prelucrate. Din acest motiv, starea rețelei nu mai poate fi redusă la o singură coadă comună, ca la modelul serie; sunt necesare locații separate care să rețină, pe de o parte, cererile aflate încă în lucru la fiecare procesor, iar pe de altă parte, rezultatele parțiale deja obținute și care așteaptă perechea [4, 6]. Momentul de reunire este modelat printr-o tranziție fără timp asociat (o tranziție *receptor*): ea devine executabilă numai când în ambele locații de rezultate parțiale există câte un jeton, iar executarea ei elimină simultan cererea din sistem [2, 6].

3. Modelul cu organizare sincronă

Modelul sincron introduce o constrângere suplimentară față de cel paralel: cele două procesoare nu mai lucrează pe cereri diferite, ci trebuie să fie disponibile *simultan* pentru a începe o nouă prelucrare. Starea sistemului trebuie așadar să surprindă două lucruri distincte — câte cereri așteaptă să fie preluate și dacă procesoarele sunt, chiar în acest moment, libere amândouă [6]. Primul aspect e reprezentat printr-o locație obișnuită de coadă, fără jetoane la pornire, întrucât sistemul începe fără cereri în așteptare [4]. Al doilea aspect are nevoie de o locație separată, care pornește cu un jeton (semnalând că, inițial, resursele sunt libere) și care se golește exact atunci când o prelucrare începe [5, 4]. Pornirea unei prelucrări este, la rândul ei, o singură tranziție care mută cererea simultan în bufferele ambelor procesoare; abia când amândouă și-au terminat sarcina, cererea este scoasă din sistem și jetonul „resurse libere” revine în locația lui, redeschizând ciclul pentru următoarea cerere [5, 6].

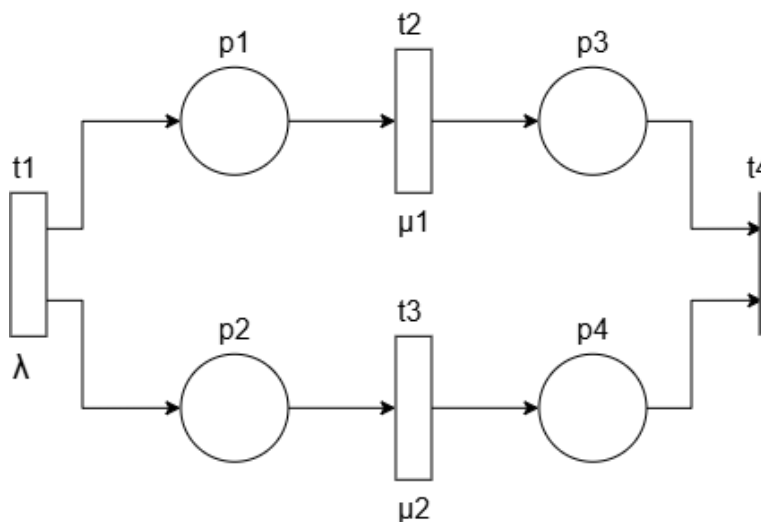


Figura 3.2: Modelul paralel al rețelei Petri

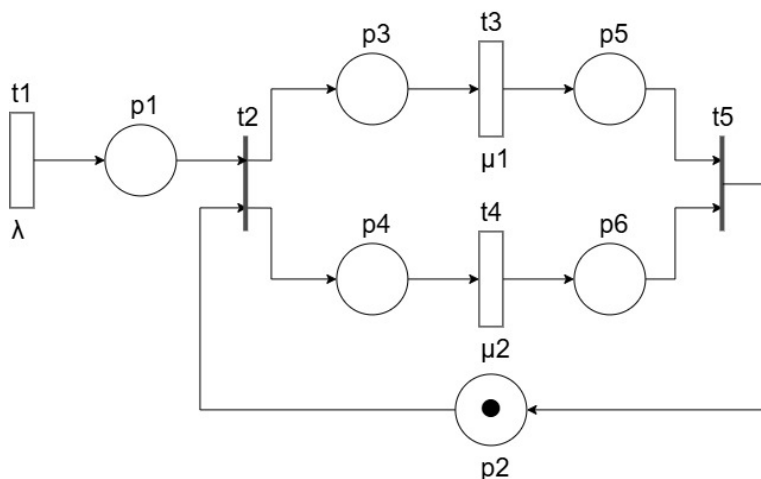


Figura 3.3: Modelul sincron al rețelei Petri

Deși cele trei arhitecturi prezentate surprind, fiecare, câte un mecanism fundamental de interacțiune între procese concurente — secvențialitate, paralelism, respectiv sincronizare — sistemele reale de servire combină adesea mai multe astfel de mecanisme simultan, iar cererile tratate nu sunt, de regulă, omogene ca tip. Contribuția originală a prezentei lucrări constă în proiectarea unui model compus, care extinde topologiile de mai sus în trei direcții:

1. se introduc patru fluxuri distincte de cereri, grupate în două perechi;
2. prima pereche de clase (culorile 1 și 2) traversează inițial o **stație comună de servire FIFO**, care le tratează în ordinea reală a sosirii, indiferent de clasă — rata de servire depinzând totuși de clasa cererii aflate în capul cozii;
3. rezultatul acestei prime servări intră, alături de a doua pereche de clase (culorile 3 și 4, sosite direct, fără nicio servire prealabilă), într-o a doua etapă cu **prioritate nepreemptivă**, la nivelul a două servere — fiecare server acceptă prioritar clasa venită din prima servire și, doar în absența acesteia, trece la procesarea clasei secundare aflate în așteptare;
4. cererile care nu beneficiază de prioritate la această a doua etapă sunt direcționate către o **a treia etapă, de sincronizare**, construită după principiul descris la modelul sincron (punctul 3

de mai sus): o singură cerere ocupă simultan ambele ramuri de procesare ale acestei etape, iar finalizarea ei este condiționată de terminarea ambelor ramuri.

Modelul rezultat permite observarea directă a modului în care trei mecanisme distincte — servirea FIFO cu rată dependentă de clasă, prioritatea nepreemptivă și sincronizarea de tip fork-join — se cumulează asupra timpului de răspuns pentru diferitele clase de cereri, aspect analizat cantitativ în Capitolul 4.

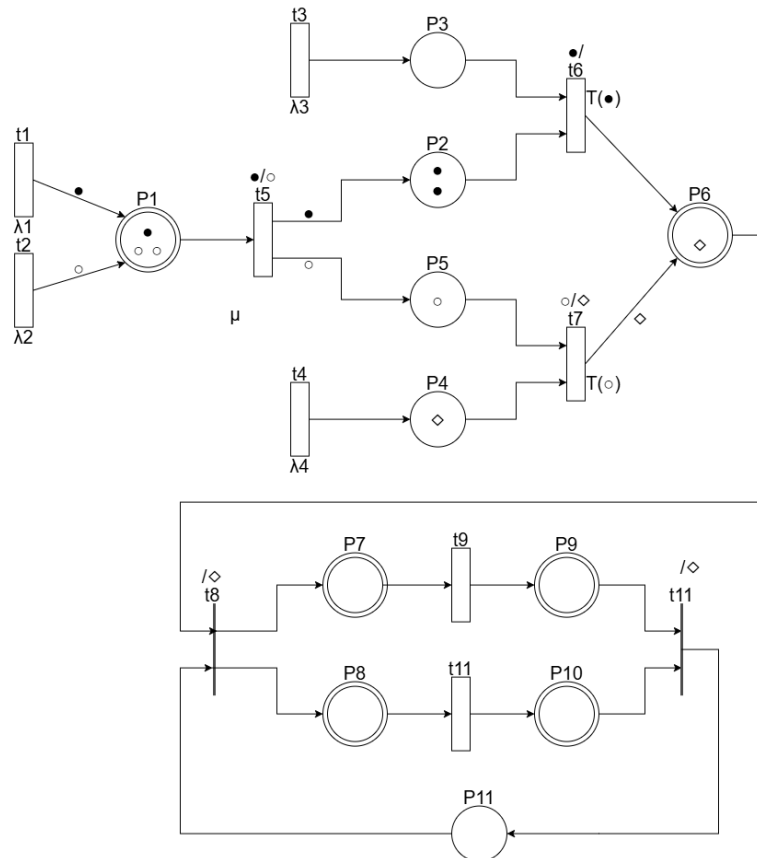


Figura 3.4: Modelul compus al rețelei Petri

Structura rețelei este organizată în trei etape. În prima etapă, cererile de culoare 1 și 2 sosesc într-o locație comună, servită în ordinea sosirii de o singură tranziție temporizată; rezultatul este direcționat, în funcție de clasa cererii tocmai servite, spre una din două locații de așteptare distincte. Cererile de culoare 3 și 4 nu parcurg această primă etapă, sosind direct în locațiile lor proprii de așteptare. În a doua etapă, fiecare din cele două servere primește cereri din două surse: una cu prioritate — provenită din prima etapă — și una secundară, sosită direct. Locațiile asociate cererilor cu prioritate sunt conectate direct la tranziția de servire a serverului respectiv, iar cele secundare intră în competiție pentru același server, fiind selectate doar în absența unei cereri prioritare în așteptare. La finalizarea acestei a doua servări, cererile prioritare părăsesc sistemul — timpul lor de răspuns fiind calculat de la momentul sosirii inițiale, incluzând așadar durata petrecută în ambele etape de servire — în timp ce cererile secundare sunt direcționate spre locația comună de așteptare a celei de-a treia etape.

3.3. Algoritmi și Pseudocod

O simulare cu evenimente discrete trebuie să decidă, la fiecare pas, ce se întâmplă și când. Pentru majoritatea tranzițiilor rețelei, răspunsul e direct: fiecăreia i se asociază o durată, eșantionată în momentul în care devine validată, iar motorul de simulare caută, la fiecare pas, tranziția a cărei durată rămasă este cea mai mică — cea care se va produce prima, cronologic. Pornirea etapei de sincronizare

și încheierea ei (Secțiunea 3.2) fac însă excepție: nu au nicio durată proprie, ci se execută instantaneu, imediat ce condiția lor devine adevărată. Din acest motiv, bucla principală a fost organizată în două faze distincte, executate la fiecare iterație:

- **rezolvarea instantanee** — pornirea celor două ramuri ale etapei de sincronizare și încheierea ei nu au nicio durată proprie și sunt executate imediat ce condiția lor devine adevărată; în aceeași fază se eșantionează și durata oricărei tranziții temporizate care tocmai a devenit validată — inclusiv aplicarea regulii de selecție, acolo unde aceasta există (servire comună FIFO sau cu prioritate) — pentru ca ea să poată intra în etapa următoare;
- **avansarea ceasului** — dintre tranzițiile temporizate deja aflate „în lucru” (sosirile, cele trei servicii de prelucrare și cele două ramuri ale etapei de sincronizare), se selectează cea a cărei durată rămasă este minimă, iar ceasul avansează direct până la acel moment.

Notății:

- $D[1..11]$: durata rămasă pentru fiecare tranziție;
- $P1, P2, P3, P4, P5, P6$: cele șase cozi ale rețelei (Secțiunea 3.2); fiecare cerere e o pereche (t, c) = moment sosire, culoare. $EXTRAGE(Q)/ADAUGA(Q, t, c)$ scot, respectiv adaugă, la capătul cozii Q .
- Sosiri ($c \in \{1, 2, 3, 4\}$): $coada[c]$ = coada de intrare ($P1$ pentru culorile 1,2; $P3$ pentru 3; $P4$ pentru 4), rată $\lambda[c]$.
- $t5$ (comun culorilor 1,2): rată $\mu_5[c]$, $c \in \{1, 2\}$; $(t5_c, t5_s)$ = culoarea/sosirea cererii aflate în servire.
- $t6, t7$ (cu prioritate): $\mu_6[c]$, $c \in \{1, 3\}$ (1 prioritar) și $\mu_7[c]$, $c \in \{2, 4\}$ (2 prioritar); $(t6_c, t6_s)$, $(t7_c, t7_s)$ analog.
- Sincronizare ($t9, t10$): rată comună $\mu_{9,10}[c]$, $c \in \{3, 4\}$; $(P7P8_c, P7P8_s)$ = culoarea/sosirea jobului curent; $P11$ = etapă liberă, $P7P8$ = job în curs; $t9, t10$ = ramură terminată.

Premise: cozi goale, $D[i] = -1$ pentru $i = 1, \dots, 11$, $ceas = 0$, $P11 = 1$, $P7P8 = 0$, $t9 = t10 = 0$.

Apel: RULEAZAMODEL(DS).

Algorithm 1 Bucla principală de simulare

```

1: function RULEAZAMODEL( $DS$ )
2:    $ceas \leftarrow 0$ ;  $D[i] \leftarrow -1$  pentru  $i = 1, \dots, 11$ ;  $D[1..4] \leftarrow \text{Exp}(\lambda[1..4])$ 
3:   while  $ceas < DS$  do
4:     RULEAZATRANZITII
5:      $t_e \leftarrow \arg \min_i D[i]$  astfel încât  $D[i] \geq 0$ ;  $t_{min} \leftarrow D[t_e]$ 
6:     if niciun  $t_e$  then oprește
7:     end if
8:     if  $ceas + t_{min} > DS$  then  $t_{min} \leftarrow DS - ceas$ 
9:     end if
10:     $ceas \leftarrow ceas + t_{min}$ 
11:     $D[i] \leftarrow D[i] - t_{min}$  pentru toți  $i$  astfel încât  $D[i] \geq 0$ 
12:    if  $ceas \geq DS$  then oprește
13:    end if
14:    EXECUTA( $t_e$ )
15:  end while
16:  return  $STr[ ], CT[ ]$ 
17: end function

```

Algorithm 2 Procesarea tranzițiilor

```
1: function RULEAZATRANZITII
2:   repeat
3:      $ok \leftarrow \text{false}$ 
4:     if  $D[5] < 0$  and  $|P1| > 0$  then
5:        $(t5\_c, t5\_s) \leftarrow \text{EXTRAGE}(P1); D[5] \leftarrow \text{EXP}(\mu_5(t5\_c)); ok \leftarrow \text{true}$ 
6:     end if
7:     if  $D[6] < 0$  and ( $|P2| > 0$  or  $|P3| > 0$ ) then
8:        $(t6\_c, t6\_s) \leftarrow \text{EXTRAGE}(P2)$  dacă  $|P2| > 0$ , altfel  $\text{EXTRAGE}(P3)$ 
9:        $D[6] \leftarrow \text{EXP}(\mu_6(t6\_c)); ok \leftarrow \text{true}$ 
10:    end if
11:    if  $D[7] < 0$  and ( $|P5| > 0$  or  $|P4| > 0$ ) then
12:       $(t7\_c, t7\_s) \leftarrow \text{EXTRAGE}(P5)$  dacă  $|P5| > 0$ , altfel  $\text{EXTRAGE}(P4)$ 
13:       $D[7] \leftarrow \text{EXP}(\mu_7(t7\_c)); ok \leftarrow \text{true}$ 
14:    end if
15:    if  $\neg P7P8$  and  $P11$  and  $|P6| > 0$  then
16:       $(P7P8\_c, P7P8\_s) \leftarrow \text{EXTRAGE}(P6)$ 
17:       $P7P8 \leftarrow \text{true}; P11 \leftarrow \text{false}; t9, t10 \leftarrow \text{false}$ 
18:       $D[9] \leftarrow \text{EXP}(\mu_{9,10}(P7P8\_c)); D[10] \leftarrow \text{EXP}(\mu_{9,10}(P7P8\_c)); ok \leftarrow \text{true}$ 
19:    end if
20:    if  $P7P8$  and  $t9$  and  $t10$  then
21:       $STr[P7P8\_c] \leftarrow STr[P7P8\_c] + (ceas - P7P8\_s); CT[P7P8\_c] \leftarrow$   

 $CT[P7P8\_c] + 1$ 
22:       $P7P8 \leftarrow \text{false}; P11 \leftarrow \text{true}; ok \leftarrow \text{true}$ 
23:    end if
24:  until  $ok = \text{false}$ 
25: end function
```

Algorithm 3 Executarea tranziției stabilite

```
1: procedure EXECUTA( $t_e$ )
2:   if  $t_e \in \{1, 2\}$  then
3:     ADAUGA( $P1, ceas, t_e$ );  $D[t_e] \leftarrow \text{Exp}(\lambda[t_e])$ 
4:   else if  $t_e = 3$  then
5:     ADAUGA( $P3, ceas$ );  $D[3] \leftarrow \text{Exp}(\lambda[3])$ 
6:   else if  $t_e = 4$  then
7:     ADAUGA( $P4, ceas$ );  $D[4] \leftarrow \text{Exp}(\lambda[4])$ 
8:   else if  $t_e = 5$  then
9:     ADAUGA( $P2, t5\_s, 1$ ) dacă  $t5\_c = 1$ , altfel ADAUGA( $P5, t5\_s, 2$ );  $D[5] \leftarrow -1$ 
10:  else if  $t_e = 6$  then
11:    if  $t6\_c = 1$  then  $STr[1] \leftarrow STr[1] + (ceas - t6\_s)$ ;  $CT[1] \leftarrow CT[1] + 1$ 
12:    else ADAUGA( $P6, t6\_s, 3$ )
13:    end if
14:     $D[6] \leftarrow -1$ 
15:  else if  $t_e = 7$  then
16:    if  $t7\_c = 2$  then  $STr[2] \leftarrow STr[2] + (ceas - t7\_s)$ ;  $CT[2] \leftarrow CT[2] + 1$ 
17:    else ADAUGA( $P6, t7\_s, 4$ )
18:    end if
19:     $D[7] \leftarrow -1$ 
20:  else if  $t_e = 9$  then
21:     $t9 \leftarrow \text{true}$ ;  $D[9] \leftarrow -1$ 
22:  else if  $t_e = 10$  then
23:     $t10 \leftarrow \text{true}$ ;  $D[10] \leftarrow -1$ 
24:  end if
25: end procedure
```

3.4. Alegerea tehnologiei

Implementarea simulatorului a fost realizată în limbajul C++, fără utilizarea unor biblioteci externe de simulare. Alegerea este motivată de:

- nevoia de control direct asupra buclei de evenimente discrete și a structurilor de date asociate marcajului rețelei, greu de obținut cu aceeași claritate folosind un framework de simulare de nivel înalt;
- continuitatea cu formalismul și stilul de implementare folosite pentru modelele fundamentale prezentate în Capitolul 2 — arhitecturile serie, paralelă și sincronă au fost verificate separat, iar modelul propus a fost construit adăugând, pe rând, fiecare mecanism nou (servire comună, prioritate, sincronizare) peste acest fundament deja funcțional;
- nevoia de a rula, în timp rezonabil, un număr mare de simulări independente pe durate extinse — ceva ce un limbaj interpretat ar fi îngreunat considerabil, dat fiind volumul de calcul implicat de generarea repetată a variabilelor aleatoare și de gestionarea cozilor.

3.5. Descrierea implementării

Implementarea simulatorului a fost realizată în limbajul C++, fără utilizarea unor biblioteci externe (motivată în Secțiunea 3.4). Structura de date centrală este vectorul duratelor $D[]$, asociat tranzițiilor temporizate ale rețelei (Secțiunea 3.3), în care:

- $D[i] = -1$ — tranziția i nu este validată;

- $D[i] = 0$ — tranziția i este executabilă imediat;
- $D[i] > 0$ — timpul rămas până la executarea tranziției i .

Tabelul 3.1 prezintă corespondența dintre locațiile și tranzițiile rețelei propuse și structurile de date folosite în implementare. Corespondența cu notațiile din Secțiunea 3.3 este directă.

Element rețea	Structură de date	Rol
<i>Etapă 1a – sosiri și prima servire (server FIFO comun)</i>		
P_1	$P1_s[], P1_c[], nP1$	Coadă FIFO comună culorilor 1 și 2; reține momentul sosirii și culoarea fiecărei cereri, în ordinea reală de sosire
t_5	$D[5], t5_c, t5_s$	Starea serverului: durată rămasă, culoarea și momentul sosirii cererii aflate în servire
<i>Etapă 1b – a doua servire, cu prioritate nepreemptivă</i>		
P_2	$P2[], nP2$	Coadă de așteptare la t_6 pentru culoarea 1 (prioritară)
P_3	$P3[], nP3$	Coadă de sosire directă la t_6 pentru culoarea 3 (secundară)
t_6	$D[6], t6_c, t6_s$	Starea serverului superior (servește prioritar din P_2 , altfel din P_3)
P_5	$P5[], nP5$	Coadă de așteptare la t_7 pentru culoarea 2 (prioritară)
P_4	$P4[], nP4$	Coadă de sosire directă la t_7 pentru culoarea 4 (secundară)
t_7	$D[7], t7_c, t7_s$	Starea serverului inferior (servește prioritar din P_5 , altfel din P_4)
<i>Etapă 2 – sincronizare fork-join</i>		
P_6	$P6_s[], P6_c[], nP6$	Coadă de așteptare pentru etapa de sincronizare, cât timp resursa e ocupată
P_7, P_8	$P7P8, P7P8_c, P7P8_s$	Jobul aflat simultan în procesare pe cele două ramuri (starea comună a fork-ului)
P_9, P_{10}	$t9, t10$	Finalizarea fiecărei ramuri; sincronizarea (t_{11}) așteaptă ambele semnale
P_{11}	$P11$	Resursa de sincronizare (jeton unic): 1 = etapă liberă, 0 = ocupată

Tabela 3.1: Corespondența dintre elementele rețelei și structurile de date ale implementării

Un aspect specific implementării este modul de calcul al timpului de răspuns pentru cererile care traversează două etape de servire (culorile 1 și 2): momentul sosirii inițiale, reținut o singură dată la intrarea în sistem, este propagat mai departe prin $t5_s$ și apoi prin $P2/P5$, astfel încât statistica finală (*ceas* – momentul sosirii) reflectă timpul total petrecut în ambele etape, nu doar în ultima dintre ele.

La nivel teoretic, modelul presupune cozi de așteptare de capacitate infinită, neexistând nicio restricție structurală asupra numărului maxim de cereri dintr-o locație. Array-urile folosite au totuși o dimensiune tehnică fixă (motivată de natura statică a limbajului C++), aleasă suficient de generoasă (100.000 de elemente) încât, la ratele de sosire și servire folosite, depășirea ei să nu apară practic în timpul unei simulări; alegerea a fost validată empiric prin rulări repetate.

Capitolul 4. Testarea soluției și rezultate experimentale

4.1. Metodologia de testare

Pentru fiecare dintre cele trei topologii analizate — modelul cu organizare în serie, modelul paralel și modelul sincron — evaluarea performanțelor s-a realizat prin variații controlate ale parametrilor de intrare ai simulării, urmărind două obiective: identificarea condițiilor de stabilitate ale sistemului și cuantificarea impactului variabilității timpilor de servire asupra timpului mediu de răspuns.

Parametrii variați sistematic au fost:

- **rata de sosire** λ , crescută progresiv spre pragul de saturație al sistemului, pentru a observa tranziția de la regim stabil la regim instabil;
- **ratele de servire** μ_1 și μ_2 ale celor două stații, testate în mai multe combinații (egale și inegale), pentru a evidenția influența dezechilibrului dintre stații asupra performanței globale;
- **durata simulării** DS , variată între 10^5 și 10^6 unități de timp, pentru a verifica convergența rezultatelor spre valori stabile în regim staționar.

Configurațiile testate formează un experiment controlat: un sistem simetric ($\mu_1 = \mu_2 = 10$), un dezechilibru major ($\mu_1 = 11, \mu_2 = 9$) și o asimetrie minoră ($\mu_1 = 11, \mu_2 = 10$). Valorile parametrului λ au fost alese diferit pentru modelul sincron față de modelele serie și paralel, pentru a ține cont de diferențele apărute la pragul de saturație.

Pentru evaluarea fiecărei configurații s-au monitorizat trei indicatori: timpul mediu de răspuns (Trm) și dimensiunea maximă atinsă de cozile de așteptare ($maxP1, maxP2$). Analiza nu s-a limitat la performanța generală indicată de medii, deoarece un Trm moderat poate ascunde vârfurile extreme de trafic și momentele de blocaj temporar ale sistemului. Urmărirea dimensiunii maxime a cozilor este esențială, deoarece, în practică, resursele fizice ale unui sistem de calcul (precum memoria alocată bufferelor) trebuie dimensionate pentru a face față celui mai rău scenariu posibil — vârful maxim de cereri — și nu doar unei încărcări medii.

Pentru a evidenția faptul că performanța unui sistem de servire depinde nu doar de capacitatea medie de procesare, ci și de predictibilitatea acesteia, fiecare topologie a fost testată în două scenarii. Primul utilizează o distribuție exponențială a timpilor de servire, $Exp(\mu_i)$, caracterizată printr-o dispersie ridicată. Al doilea scenariu folosește o distribuție normală, $N(1/\mu_i, \sigma_i)$, care păstrează aceeași medie a timpului de execuție ($1/\mu_i$), dar limitează fluctuațiile prin fixarea abaterii standard la o valoare redusă, $\sigma_i = 1/(5\mu_i)$. Această constrângere a variațiilor haotice permite izolarea clară a impactului pe care îl are variabilitatea timpului de servire asupra stabilității globale a sistemului, aspect analizat pe larg în Secțiunea 4.5.

Toate simulările au fost rulate pornind de la un sistem fără cereri în așteptare, motiv pentru care, prima etapă a execuției nu reflectă performanța stabilă a sistemului. Pentru a preveni distorsionarea rezultatelor finale de către această perioadă de inițializare, experimentele au fost rulate pe durate foarte mari. În acest fel, impactul măsurătorilor de la început este minimizat, iar acuratețea datelor finale este demonstrată prin analiza de convergență din Secțiunea 4.3.

4.2. Modelul în serie

Configurația testată folosește cele trei combinații ale ratelor de servire stabilite în metodologie (Secțiunea 4.1) — $\mu_1 = 11, \mu_2 = 9$; $\mu_1 = 10, \mu_2 = 10$; $\mu_1 = 11, \mu_2 = 10$ — cu rata de sosire λ variată de la 7 până la pragul de saturație propriu fiecărei configurații, folosind o durată de simulare $DS = 10^6$.

Pragul teoretic de stabilitate, $\lambda < \min(\mu_1, \mu_2)$, este identic cu cel al modelului paralel (Secțiunea 4.3) — motiv pentru care gradul de încărcare $\rho = \lambda / \min(\mu_1, \mu_2)$ ia aceleași valori la ambele modele, pentru aceiași parametri.

λ	$\rho (\mu_1 = 11, \mu_2 = 9)$	$\rho (\mu_1 = \mu_2 = 10)$	$\rho (\mu_1 = 11, \mu_2 = 10)$
7	0,78	0,70	0,70
8	0,89	0,80	0,80
9	1,00*	0,90	0,90
9,5	1,06*	0,95	0,95
9,7	1,08*	0,97	0,97
9,9	1,10*	0,99	0,99
10	1,11*	1,00*	1,00*

Tabela 4.1: Gradul de încărcare $\rho = \lambda / \min(\mu_1, \mu_2)$ pentru modelul serie (* = instabil, $\rho \geq 1$)

	$\mu_1 = 11, \mu_2 = 9$			$\mu_1 = 10, \mu_2 = 10$			$\mu_1 = 11, \mu_2 = 10$		
	<i>Trm</i>	<i>maxP1</i>	<i>maxP2</i>	<i>Trm</i>	<i>maxP1</i>	<i>maxP2</i>	<i>Trm</i>	<i>maxP1</i>	<i>maxP2</i>
$\lambda = 7$	0,7486	36	57	0,6658	42	36	0,5816	33	44
$\lambda = 8$	1,3324	41	119	1,0027	58	64	0,8297	43	54
$\lambda = 9$	113,159	59	2810	1,9922	107	112	1,5087	59	194
$\lambda = 9,5$	—	—	—	3,8953	216	197	2,6813	81	205
$\lambda = 9,7$	—	—	—	6,3735	297	432	4,0685	94	308
$\lambda = 9,9$	—	—	—	20,8368	709	712	11,2027	123	813
$\lambda = 10$	—	—	—	213,88	3401	5111	111,058	115	3914

Tabela 4.2: Rezultatele obținute pentru modelul serie, repartiție exponențială ($DS = 10^6$)

În configurația asimetrică ($\mu_1 = 11, \mu_2 = 9$), stația 2 devine principalul punct critic de congestie al sistemului. Pe măsură ce λ crește de la 7 la 9, valoarea $maxP_2$ crește brusc de la 57 la 2810 — atingând pragul de stabilitate, $\min(\mu_1, \mu_2) = 9$ — în timp ce $maxP_1$ rămâne la un nivel redus (36 până la 59). Pentru $\lambda \geq 9,5$, sistemul intră în regim de instabilitate, cozile nemaiputând atinge o stare staționară.

În cazul simetric ($\mu_1 = \mu_2 = 10$), pragul de stabilitate este $\lambda < 10$ — același ca la configurația $\mu_1 = 11, \mu_2 = 10$. Rularea până la $\lambda = 10$ (exact la prag) ilustrează comportamentul sistemului aflat la limită: cozile ating deja valori mari, dar comparabile între cele două stații ($maxP_1 = 3401$, $maxP_2 = 5111$), spre deosebire de configurația asimetrică, unde congestia se concentrează aproape exclusiv la stația mai lentă.

Pentru configurația cu asimetrie redusă ($\mu_1 = 11, \mu_2 = 10$), pragul de stabilitate este tot $\lambda < 10$. Coada stației 2 concentrează cea mai mare parte a saturației sistemului (atingând 3914 la $\lambda = 10$), în timp ce prima stație își menține un nivel de încărcare relativ stabil, $maxP_1$ plafonându-se în intervalul 100–120 începând cu $\lambda = 9$. O analiză comparativă detaliată a acestor comportamente, față de modelele paralel și sincron, este realizată în Secțiunea 4.5.

	$\mu_1 = 11, \mu_2 = 9$			$\mu_1 = 10, \mu_2 = 10$			$\mu_1 = 11, \mu_2 = 10$		
	<i>Trm</i>	<i>maxP1</i>	<i>maxP2</i>	<i>Trm</i>	<i>maxP1</i>	<i>maxP2</i>	<i>Trm</i>	<i>maxP1</i>	<i>maxP2</i>
$\lambda = 7$	0,41	24	25	0,3518	22	9	0,3238	22	14
$\lambda = 8$	0,6786	27	49	0,4591	32	13	0,4148	26	23
$\lambda = 9$	519,311	36	7847	0,7845	57	26	0,6824	37	61
$\lambda = 9,5$	–	–	–	1,4486	119	47	1,2139	51	102
$\lambda = 9,7$	–	–	–	2,3054	159	70	1,9189	54	176
$\lambda = 9,9$	–	–	–	7,0814	646	124	5,6004	61	480
$\lambda = 10$	–	–	–	169,747	3547	665	200,696	73	6872

Tabela 4.3: Rezultatele obținute pentru modelul serie, repartiție normală ($DS = 10^6$)

Comparând cele două seturi de date, se observă că limitarea dispersiei timpilor de servire nu modifică pragul teoretic de stabilitate (rămâne $\min(\mu_1, \mu_2)$), dar reduce considerabil timpul mediu de răspuns și dimensiunea cozilor în regimul stabil — același efect observat și la modelul paralel.

4.3. Modelul paralel

Pentru modelul paralel s-au testat aceleași trei combinații ale ratelor de servire stabilite în metodologie (Secțiunea 4.1) — $\mu_1 = 11, \mu_2 = 9$; $\mu_1 = 10, \mu_2 = 10$; $\mu_1 = 11, \mu_2 = 10$ — cu rata de sosire λ variată de la 7 până la pragul de saturatie propriu fiecărei configurații, folosind o durată de simulare $DS = 10^6$.

Deoarece în modelul paralel fiecare cerere ocupă simultan ambele stații, iar sincronizarea are loc doar la ieșirea din sistem, cele două stații se comportă, din punctul de vedere al propriei cozi de intrare, ca două sisteme M/M/1 independente, alimentate de același flux λ . Pragul de stabilitate al întregului sistem este dat, prin urmare, de stația *mai lentă*: $\lambda < \min(\mu_1, \mu_2)$ — identic cu pragul modelului serie (Secțiunea 4.2).

λ	$\rho (\mu_1 = 11, \mu_2 = 9)$	$\rho (\mu_1 = \mu_2 = 10)$	$\rho (\mu_1 = 11, \mu_2 = 10)$
7	0,78	0,70	0,70
8	0,89	0,80	0,80
9	1,00*	0,90	0,90
9,5	1,06*	0,95	0,95
9,7	1,08*	0,97	0,97
9,9	1,10*	0,99	0,99
10	1,11*	1,00*	1,00*

Tabela 4.4: Gradul de încărcare $\rho = \lambda / \min(\mu_1, \mu_2)$ pentru modelul paralel (* = instabil, $\rho \geq 1$)

	$\mu_1 = 11, \mu_2 = 9$			$\mu_1 = 10, \mu_2 = 10$			$\mu_1 = 11, \mu_2 = 10$		
	<i>Trm</i>	<i>maxP1</i>	<i>maxP2</i>	<i>Trm</i>	<i>maxP1</i>	<i>maxP2</i>	<i>Trm</i>	<i>maxP1</i>	<i>maxP2</i>
$\lambda = 7$	0,5594	34	60	0,4701	39	40	0,4169	35	38
$\lambda = 8$	1,0471	50	93	0,6994	58	63	0,5966	50	60
$\lambda = 9$	177,189	61	8281	1,3905	124	147	1,109	71	108
$\lambda = 9,5$	–	–	–	2,781	187	211	2,1055	87	201
$\lambda = 9,7$	–	–	–	4,5155	280	267	3,3199	82	301
$\lambda = 9,9$	–	–	–	12,8357	635	682	9,9037	122	704
$\lambda = 10$	–	–	–	270,965	5210	7047	168,411	116	4084

Tabela 4.5: Rezultatele obținute pentru modelul paralel, repartiție exponențială ($DS = 10^6$)

Acest fapt este vizibil direct în date. Pentru configurația $\mu_1 = 11, \mu_2 = 9$, coada stației 2 (mai lentă) explodează deja la $\lambda = 9$ ($maxP_2 = 8281$), în timp ce coada stației 1 (mai rapidă, cu rezervă de capacitate) rămâne relativ modestă ($maxP_1 = 61$) — o diferență de peste 100 de ori între cele două cozi, la aceeași rată de sosire. Aceeași asimetrie nu mai apare la configurația $\mu_1 = 10, \mu_2 = 10$, unde cele două stații, având aceeași capacitate, ajung la valori comparabile ale cozii maxime pe măsură ce λ se apropie de 10. Concluzia practică este directă: performanța globală a unui sistem paralel cu stații inegale este limitată strict de stația cea mai lentă, indiferent cât de rapidă este cealaltă.

	$\mu_1 = 11, \mu_2 = 9$			$\mu_1 = 10, \mu_2 = 10$			$\mu_1 = 11, \mu_2 = 10$		
	<i>Trm</i>	<i>maxP1</i>	<i>maxP2</i>	<i>Trm</i>	<i>maxP1</i>	<i>maxP2</i>	<i>Trm</i>	<i>maxP1</i>	<i>maxP2</i>
$\lambda = 7$	0,3158	21	33	0,2448	27	28	0,2279	19	23
$\lambda = 8$	0,5738	25	52	0,3414	32	35	0,3131	24	29
$\lambda = 9$	128,768	36	3142	0,6302	57	60	0,5747	39	63
$\lambda = 9,5$	–	–	–	1,2091	120	114	1,0781	60	129
$\lambda = 9,7$	–	–	–	1,9399	172	170	1,7556	60	163
$\lambda = 9,8$	–	–	–	5,6133	332	347	5,3154	56	485
$\lambda = 10$	–	–	–	369,979	6163	6419	214,026	69	5376

Tabela 4.6: Rezultatele obținute pentru modelul paralel, repartiție normală ($DS = 10^6$)

Ca și la modelul serie, limitarea dispersiei timpilor de servire nu modifică pragul teoretic de stabilitate, dar reduce sensibil timpul mediu de răspuns și dimensiunea cozilor.

4.4. Modelul sincron

Pentru modelul sincron s-au păstrat aceleași trei combinații ale ratelor de servire ($\mu_1 = 11, \mu_2 = 9$; $\mu_1 = 10, \mu_2 = 10$; $\mu_1 = 11, \mu_2 = 10$), folosite și la modelele serie și paralel, pentru comparabilitate directă în Secțiunea 4.5.

Intuitiv, s-ar putea presupune că pragul de stabilitate rămâne același ca la modelul paralel, $\lambda < \min(\mu_1, \mu_2)$, atâta timp cât fiecare stație, luată separat, poate procesa fluxul de cereri. Rezultatele experimentale infirmă însă această presupunere: sistemul devine instabil la valori ale lui λ semnificativ mai mici decât $\min(\mu_1, \mu_2)$. Cauza este constrângerea de sincronizare: o cerere nouă nu poate intra în procesare decât după ce ambele ramuri ale cererii curente s-au încheiat, adică timpul efectiv ocupat de o cerere în sistem nu este $1/\mu_1$ sau $1/\mu_2$, ci maximum celor două: $S = \max(T_1, T_2)$, cu $T_1 \sim \text{Exp}(\mu_1)$, $T_2 \sim \text{Exp}(\mu_2)$ independente.

Media acestui maxim se calculează pornind de la identitatea $T_1 + T_2 = \max(T_1, T_2) + \min(T_1, T_2)$, valabilă pentru orice două variabile aleatoare, și de la faptul că minimum a două exponențiale indepen-

dente este tot exponențial, cu rata sumă: $\min(T_1, T_2) \sim \text{Exp}(\mu_1 + \mu_2)$. Rezultă:

$$E[S] = E[T_1] + E[T_2] - E[\min(T_1, T_2)] = \frac{1}{\mu_1} + \frac{1}{\mu_2} - \frac{1}{\mu_1 + \mu_2}.$$

Rata efectivă de servire a sistemului, $\mu_{ef} = 1/E[S]$, este deci întotdeauna mai mică decât $\min(\mu_1, \mu_2)$ — sistemul este penalizat de timpul în care ramura mai rapidă a terminat deja, dar trebuie să aștepte ramura mai lentă. Pentru $\mu_1 = 11$, $\mu_2 = 9$: $\mu_{ef} = 6,578$; pentru $\mu_1 = \mu_2 = 10$: $\mu_{ef} \approx 6,667$; pentru $\mu_1 = 11$, $\mu_2 = 10$: $\mu_{ef} = 6,979$. În consecință, rata de sosire λ a fost variată pe intervalul 4–7, în loc de intervalul 7–10 folosit la serie și paralel, pentru a surprinde atât regimul stabil, cât și tranziția spre instabilitate în vecinătatea lui μ_{ef} .

λ	$\rho(\mu_1 = 11, \mu_2 = 9)$	$\rho(\mu_1 = \mu_2 = 10)$	$\rho(\mu_1 = 11, \mu_2 = 10)$
4	0,61	0,60	0,57
5	0,76	0,75	0,72
6	0,91	0,90	0,86
6,3	0,96	0,95	0,90
6,58	1,00*	0,99	0,94
6,67	1,01*	1,00*	0,96
6,98	1,06*	1,05*	1,00*

Tabela 4.7: Gradul de încărcare $\rho = \lambda/\mu_{ef}$ pentru modelul sincron, repartiție exponențială (* = instabil, $\rho \geq 1$)

Alegerea $\lambda \in \{4; 5; 6; 6,3; 6,58; 6,67; 6,98\}$ oferă, pentru fiecare configurație, mai multe puncte clar stabile și trei puncte care marchează exact pragul teoretic al fiecărei configurații — instabilitatea apărând decalat, în ordinea μ_{ef} calculată pentru fiecare.

	$\mu_1 = 11, \mu_2 = 9$		$\mu_1 = 10, \mu_2 = 10$		$\mu_1 = 11, \mu_2 = 10$	
	<i>Trm</i>	<i>maxP</i>	<i>Trm</i>	<i>maxP</i>	<i>Trm</i>	<i>maxP</i>
$\lambda = 4$	0,3354	22	0,3246	22	0,2933	22
$\lambda = 5$	0,5295	37	0,4987	38	0,4251	31
$\lambda = 6$	1,3785	82	1,1976	80	0,8226	55
$\lambda = 6,3$	2,8185	186	2,1398	154	1,1801	100
$\lambda = 6,58$	375,05	6164	8,7067	383	1,9591	139
$\lambda = 6,67$	—	—	640,438	7749	2,5006	152
$\lambda = 6,98$	—	—	—	—	422,407	5891

Tabela 4.8: Rezultatele obținute pentru modelul sincron, repartiție exponențială ($DS = 10^6$)

Rezultatele confirmă tendința anticipată: sistemul sincron devine instabil semnificativ mai devreme decât modelul paralel, la valori ale lui λ apropiate de μ_{ef} , nu de $\min(\mu_1, \mu_2)$. La configurația $\mu_1 = 11$, $\mu_2 = 9$, sistemul rămâne stabil până la $\lambda = 6,3$, cu $Trm = 2,8185$, însă la atingerea pragului teoretic $\lambda = 6,58$ performanța se degradează masiv ($Trm = 375,05$). Tendința este similară la celelalte două configurații, cu pragul de instabilitate deplasat corespunzător mai sus, în acord cu

valorile mai mari ale lui μ_{ef} calculate pentru acestea. Comparativ cu modelele serie și paralel, la rate de sosire aparent sigure (mult sub capacitatea individuală a stațiilor), timpul mediu de răspuns suferă penalizări severe — diferență atribuibilă exclusiv mecanismului de sincronizare, care expune sistemul la fluctuațiile maxime de execuție ale ambelor ramuri.

Dacă în cazul variabilelor exponențiale calculul timpului mediu de sincronizare rezultă imediat din proprietățile distribuției, variabilele normale necesită o abordare analitică diferită. Pentru a depăși lipsa unei forme închise simple, calculul mediei maximului a două variabile normale independente a fost efectuat folosind ecuațiile derivate de Clark [3], o soluție clasică din cercetarea operațională destinată problemelor de acest tip.

$$E[\max(T_1, T_2)] = m_1 \Phi(a) + m_2 \Phi(-a) + \theta \varphi(a), \quad \theta = \sqrt{\sigma_1^2 + \sigma_2^2}, \quad a = \frac{m_1 - m_2}{\theta},$$

unde Φ și φ sunt funcția de repartiție, respectiv densitatea normalei standard, iar $m_i = 1/\mu_i$. Aplicând formula, rezultă praguri de stabilitate (μ_{ef}) mult mai ridicate: 8,680 pentru $\mu_1 = 11, \mu_2 = 9$; 8,986 pentru $\mu_1 = \mu_2 = 10$; 9,360 pentru $\mu_1 = 11, \mu_2 = 10$.

λ	$\rho (\mu_1 = 11, \mu_2 = 9)$	$\rho (\mu_1 = \mu_2 = 10)$	$\rho (\mu_1 = 11, \mu_2 = 10)$
4	0,46	0,45	0,43
5	0,58	0,56	0,53
6	0,69	0,67	0,64
7	0,81	0,78	0,75
8	0,92	0,89	0,85
8,5	0,98	0,95	0,91
8,68	1,00*	0,97	0,93
8,99	1,04*	1,00*	0,96
9,36	1,08*	1,04*	1,00*

Tabela 4.9: Gradul de încărcare $\rho = \lambda/\mu_{ef}$ pentru modelul sincron, repartiție normală (* = instabil, $\rho \geq 1$)

Alegerea $\lambda \in \{4; 5; 6; 7; 8; 8,5; 8,68; 8,99; 9,36\}$ oferă, pentru fiecare configurație, mai multe puncte de încărcare treptată (stabile) și trei puncte care marchează exact pragul teoretic de instabilitate specific distribuției normale. La fel ca în modelul exponențial, instabilitatea apare decalat, în ordinea valorilor μ_{ef} deduse pentru fiecare caz, însă la rate de sosire substanțial mai ridicate, reflectând performanța superioară a modelului cu dispersie controlată.

	$\mu_1 = 11, \mu_2 = 9$		$\mu_1 = 10, \mu_2 = 10$		$\mu_1 = 11, \mu_2 = 10$	
	<i>Trm</i>	<i>maxP</i>	<i>Trm</i>	<i>maxP</i>	<i>Trm</i>	<i>maxP</i>
$\lambda = 4$	0,1656	10	0,1570	11	0,1475	11
$\lambda = 5$	0,1857	14	0,1827	15	0,1695	13
$\lambda = 6$	0,2477	21	0,2258	20	0,2045	19
$\lambda = 7$	0,3609	31	0,3123	34	0,2690	24
$\lambda = 8$	0,8092	72	0,5727	76	0,4279	45
$\lambda = 8,5$	2,8913	198	1,1139	127	0,6502	74
$\lambda = 8,68$	367,705	5918	1,7611	215	0,8066	76
$\lambda = 8,99$	—	—	486,062	8120	1,4309	119
$\lambda = 9,36$	—	—	—	—	557,524	7625

Tabela 4.10: Rezultatele obținute pentru modelul sincron, repartiție normală ($DS = 10^6$)

Capitolul 5. Concluzii și dezvoltări ulterioare

5.1. Concluzii generale

Lucrarea de față a abordat problema evaluării performanțelor în sisteme de servire cu topologie complexă, utilizând formalismul Rețelelor Petri stohastice. Obiectivul principal, acela de a dezvolta un simulator software complet funcțional pentru analiza regimului tranzitoriu și staționar al acestor arhitecturi, a fost atins cu succes prin implementarea unei soluții proprii în limbajul C++. Metodologia bazată pe simularea cu evenimente discrete a permis o analiză detaliată a modului în care interacțiunea dintre resurse și variabilitatea timpilor de execuție influențează parametrii globali ai sistemului.

În urma experimentelor, s-a observat că performanța și pragul de stabilitate ale unui sistem nu depind exclusiv de capacitatea individuală de procesare a serverelor, ci sunt profund condiționate de arhitectura topologică. Astfel, modelele cu procesare independentă (în serie și paralel) își mențin stabilitatea atâta timp cât rata de sosire nu depășește capacitatea celei mai lente stații ($\min(\mu_1, \mu_2)$). Totuși, modul în care gestionează congestia diferă: în timp ce modelul paralel izolează aglomerarea strict pe ramura mai lentă (permițând ramurii rapide să funcționeze optim), modelul în serie propagă inevitabil întârzierile de-a lungul întregului flux de prelucrare.

Pe de altă parte, modelul sincron a demonstrat o vulnerabilitate accentuată. Deoarece finalizarea unei sarcini depinde de terminarea simultană a mai multor procese paralele, timpul efectiv de execuție este dictat de valoarea maximă a duratelor individuale. Acest comportament penalizează sever rata efectivă de servire (μ_{ef}) a sistemului, demonstrând că arhitecturile cu sincronizare cedează mult mai repede sub sarcină și necesită rezerve de capacitate semnificativ mai mari pentru a evita blocajele.

Nu în ultimul rând, variabilitatea timpilor de servire a jucat un rol crucial în comportamentul fiecărui modul. Limitarea fluctuațiilor prin utilizarea distribuției normale a condus la performanțe vizibil superioare în toate topologiile comparativ cu distribuția exponențială, dovedind că, în proiectarea sistemelor cu evenimente discrete, predictibilitatea execuției este la fel de importantă precum viteza brută de procesare.

5.2. Sinteza rezultatelor și contribuții personale

Prin testarea topologiilor clasice (serie, paralel, sincron) și a modelului arhitectural compus, s-au desprins următoarele concluzii esențiale:

- **Impactul topologiei:** Modelul sincron atinge pragul de instabilitate la rate de sosire semnificativ mai mici comparativ cu modelele serie și paralel, demonstrând vulnerabilitatea mecanismelor de tip sincronizare la decalajele de procesare.
- **Efectul variabilității:** Limitarea dispersiei timpilor de servire (prin utilizarea distribuției normale în locul celei exponențiale) a condus la o reducere drastică a timpului mediu de răspuns și a cozilor maxime de așteptare, subliniind importanța predictibilității în sistemele concurente.
- **Modelul arhitectural compus:** Implementarea modelului extins, ce integrează servirea FIFO dependentă de clasă, prioritățile nepreemptive și etapele de sincronizare, reprezintă o contribuție majoră. Aceasta a validat funcționalitatea simulatorului pe scenarii apropiate de sistemele reale.
- **Stabilitatea algoritmului:** Execuția simulărilor pe durate extinse (peste 10^5 și 10^6 unități de timp) a demonstrat fiabilitatea structurilor de date personalizate și a gestionării dinamice a cozilor de așteptare.

Bibliografie

- [1] N. V. Nenkov and E. Z. Spasova, "Implementation of a Neural Network Using Simulator and Petri Nets," *International Journal of Advanced Computer Science and Applications*, Vol. 7, No. 1, 2016, DOI: 10.14569/IJACSA.2016.070155.
- [2] M. Ajmone Marsan, "Stochastic Petri Nets: An Elementary Introduction," in *Advances in Petri Nets 1989*, *Lecture Notes in Computer Science*, vol. 424, Springer, 1990, pp. 1–29, DOI: 10.1007/3-540-52494-0_23.
- [3] C. E. Clark, "The Greatest of a Finite Set of Random Variables," *Operations Research*, vol. 9, no. 2, pp. 145–162, 1961, DOI: 10.1287/opre.9.2.145.
- [4] J. L. Peterson, *Petri Net Theory and the Modeling of Systems*, Englewood Cliffs, NJ: Prentice-Hall, 1981.
- [5] T. Murata, "Petri Nets: Properties, Analysis and Applications," *Proceedings of the IEEE*, vol. 77, no. 4, pp. 541–580, 1989, DOI: 10.1109/5.24143.
- [6] Cașcaval, P., *Modelarea și simularea sistemelor cu evenimente discrete*, Editura Performantica, Iași, 2006.
- [7] Trivedi, K.S., *Probability and Statistics with Reliability, Queueing, and Computer Science Applications*, John Wiley & Sons, New York, 2002.
- [8] Kendall, D.G., *Stochastic Processes Occurring in the Theory of Queues and their Analysis by the Method of the Imbedded Markov Chain*, *The Annals of Mathematical Statistics*, vol. 24, nr. 3, pp. 338–354, 1953.